

Let's delve deep into each of these Java concepts with detailed explanations and illustrative examples.

Session 11: Java Deep Dive

1. Access Modifiers

Access modifiers in Java control the visibility and accessibility of classes, fields (variables), methods, and constructors. They are crucial for encapsulation, a core OOP principle, by preventing unauthorized access and modification of data.

a. public

- **Visibility:** Accessible from anywhere. This includes within the same class, same package, subclasses (in the same or different packages), and entirely different packages.
- **Usage:** Typically used for methods that provide an interface to the class, or for fields that are meant to be widely accessible (though this is less common for fields due to encapsulation best practices).

Example:

Java

```
// File: com/example/library/Book.java
```

```
package com.example.library;
```

```
public class Book {
```

```
    public String title; // Accessible from anywhere
```

```
    public String author;
```

```
    public void displayBookDetails() { // Accessible from anywhere
```

```
        System.out.println("Title: " + title + ", Author: " + author);
```

```
    }
```

```
}
```

```
// File: com/example/app/MyApp.java
```

```
package com.example.app;
```

```
import com.example.library.Book;
```

```
public class MyApp {
```

```

public static void main(String[] args) {
    Book myBook = new Book();
    myBook.title = "The Great Gatsby";
    myBook.author = "F. Scott Fitzgerald";
    myBook.displayBookDetails(); // Accessing public method
}
}

```

b. private

- **Visibility:** Accessible only within the *same class*.
- **Usage:** Primarily used for fields to enforce encapsulation. This means the internal state of an object can only be manipulated through its public methods, providing control and preventing direct external modification. It can also be used for helper methods that are only relevant internally.

Example:

Java

// File: com/example/accounts/BankAccount.java

```
package com.example.accounts;
```

```

public class BankAccount {
    private double balance; // Only accessible within BankAccount
    private String accountNumber; // Only accessible within BankAccount

```

```

    public BankAccount(String accountNumber, double initialBalance) {
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
    }

```

```

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: " + amount + ", New Balance: " + balance);
        } else {
            System.out.println("Deposit amount must be positive.");
        }
    }
}

```

```

public void withdraw(double amount) {
    if (amount > 0 && balance >= amount) {
        balance -= amount;
        System.out.println("Withdrew: " + amount + ", New Balance: " + balance);
    } else {
        System.out.println("Invalid withdrawal amount or insufficient balance.");
    }
}

```

```

// Public method to provide controlled access to the private balance
public double getBalance() {
    return balance;
}

```

```

public String getAccountNumber() {
    return accountNumber;
}
}

```

```

// File: com/example/app/BankApp.java

```

```

package com.example.app;

```

```

import com.example.accounts.BankAccount;

```

```

public class BankApp {
    public static void main(String[] args) {
        BankAccount myAccount = new BankAccount("12345", 1000.0);
        // myAccount.balance = 500; // ERROR: Cannot access private field
        myAccount.deposit(200);
        myAccount.withdraw(50);
        System.out.println("Current balance: " + myAccount.getBalance());
    }
}

```

c. protected

- **Visibility:** Accessible within the *same package* and by *subclasses* (even if the subclass is in a different package).
- **Usage:** Useful when you want to allow subclasses to access certain members, but still restrict access from unrelated classes outside the package. This supports inheritance by allowing controlled access to parent class members.

Example:

Java

```
// File: com/example/vehicles/Vehicle.java
```

```
package com.example.vehicles;
```

```
public class Vehicle {
```

```
    protected String brand; // Accessible within package and by subclasses
```

```
    protected int year;
```

```
    public Vehicle(String brand, int year) {
```

```
        this.brand = brand;
```

```
        this.year = year;
```

```
    }
```

```
    protected void displayVehicleInfo() { // Accessible within package and by subclasses
```

```
        System.out.println("Brand: " + brand + ", Year: " + year);
```

```
    }
```

```
}
```

```
// File: com/example/vehicles/Car.java (Same package)
```

```
package com.example.vehicles;
```

```
public class Car extends Vehicle {
```

```
    private int numberOfDoors;
```

```
    public Car(String brand, int year, int numberOfDoors) {
```

```
        super(brand, year);
```

```
        this.numberOfDoors = numberOfDoors;
```

```
    }
```

```
    public void displayCarDetails() {
```

```
        displayVehicleInfo(); // Accessing protected method from superclass in same package
```

```
        System.out.println("Number of Doors: " + numberOfDoors);
```

```
    }
```

```
}
```

```
// File: com/example/app/Truck.java (Different package, but a subclass)
```

```
package com.example.app;
```

```
import com.example.vehicles.Vehicle;

public class Truck extends Vehicle {
    private double cargoCapacity;

    public Truck(String brand, int year, double cargoCapacity) {
        super(brand, year);
        this.cargoCapacity = cargoCapacity;
    }

    public void displayTruckDetails() {
        displayVehicleInfo(); // Accessing protected method from superclass in different package (as a subclass)
        System.out.println("Cargo Capacity: " + cargoCapacity + " tons");
        System.out.println("Truck brand (protected access): " + brand); // Accessing protected field
    }
}

// File: com/example/app/MainApp.java
package com.example.app;

import com.example.vehicles.Vehicle;
import com.example.vehicles.Car;

public class MainApp {
    public static void main(String[] args) {
        Car myCar = new Car("Toyota", 2020, 4);
        myCar.displayCarDetails();

        Truck myTruck = new Truck("Ford", 2018, 5.0);
        myTruck.displayTruckDetails();

        // Vehicle someVehicle = new Vehicle("Honda", 2022);
        // someVehicle.displayVehicleInfo(); // ERROR: Cannot access protected method outside package and not a subclass
        // System.out.println(someVehicle.brand); // ERROR: Cannot access protected field outside package and not a subclass
    }
}
```

d. default (Package-Private)

- **Visibility:** Accessible only within the *same package*. If no access modifier is explicitly

specified, a member or class has default access.

- **Usage:** Useful for classes or members that are intended to be part of an internal implementation detail of a package and should not be exposed to the outside world.

Example:

Java

```
// File: com/example/utility/StringUtils.java
```

```
package com.example.utility;
```

```
class StringUtils { // Default access: only accessible within com.example.utility
```

```
    static String capitalize(String text) { // Default access: only accessible within com.example.utility
```

```
        if (text == null || text.isEmpty()) {
```

```
            return text;
```

```
        }
```

```
        return Character.toUpperCase(text.charAt(0)) + text.substring(1).toLowerCase();
```

```
    }
```

```
}
```

```
// File: com/example/utility/TextProcessor.java (Same package)
```

```
package com.example.utility;
```

```
public class TextProcessor {
```

```
    public void processAndPrint(String input) {
```

```
        String capitalizedText = StringUtils.capitalize(input); // Accessing default method within the same package
```

```
        System.out.println("Processed text: " + capitalizedText);
```

```
    }
```

```
}
```

```
// File: com/example/app/MyTextApp.java (Different package)
```

```
package com.example.app;
```

```
import com.example.utility.TextProcessor;
```

```
// import com.example.utility.StringUtils; // ERROR: StringUtils has default access, not visible here
```

```
public class MyTextApp {
```

```
    public static void main(String[] args) {
```

```
        TextProcessor processor = new TextProcessor();
```

```
processor.processAndPrint("hello world");

// StringUtils.capitalize("test"); // ERROR: Cannot access default class/method from a different
package
}
}
```

2. Packages and Import Statements

a. Packages

- **Purpose:** Packages are a mechanism to organize classes, interfaces, and sub-packages into a logical hierarchy. They help in:
 - **Avoiding Naming Collisions:** Two classes can have the same name if they are in different packages (e.g., `java.util.List` and `java.awt.List`).
 - **Controlling Access:** As seen with default and protected access modifiers.
 - **Modularity:** Grouping related classes together for better maintainability and reusability.
- **Declaration:** A package is declared using the `package` keyword as the *first statement* in a Java source file.
- **Naming Convention:** Packages are usually named in reverse domain name order (e.g., `com.example.myapp.utilities`). All letters are typically lowercase.

Example:

Java

```
// File: com/mycompany/myapp/models/User.java
package com.mycompany.myapp.models; // Package declaration

public class User {
    private String username;
    private String email;

    public User(String username, String email) {
        this.username = username;
        this.email = email;
    }

    public String getUsername() {
        return username;
    }
}
```

```

    }

    public String getEmail() {
        return email;
    }
}

```

b. Import Statements

- **Purpose:** import statements allow you to refer to classes, interfaces, or static members by their simple names without having to specify their fully qualified names (package.className) every time.
- **Syntax:**
 - import packageName.ClassName; (Imports a specific class)
 - import packageName.*; (Imports all classes in a package – **not recommended for production code as it can lead to name conflicts and make code less readable**).

Example:

Java

```

// File: com/mycompany/myapp/MainApp.java
package com.mycompany.myapp;

import com.mycompany.myapp.models.User; // Importing the User class

// Another common import:
import java.util.ArrayList; // Importing a class from Java's standard library

public class MainApp {
    public static void main(String[] args) {
        User user1 = new User("john_doe", "john@example.com"); // Using simple name
        System.out.println("Username: " + user1.getUsername());

        ArrayList<String> names = new ArrayList<>(); // Using simple name
        names.add("Alice");
        System.out.println("Names: " + names);
    }
}

```


3. Static Imports

- **Purpose:** Static imports allow you to import static members (fields and methods) of a class directly, so you don't have to qualify them with the class name.
- **Syntax:**
 - `import static packageName.ClassName.staticMember;` (Imports a specific static member)
 - `import static packageName.ClassName.*;` (Imports all static members of a class)

Example:

Java

```
// File: com/example/math/Calculator.java
package com.example.math;

public class Calculator {
    public static final double PI = 3.14159; // Static field

    public static int add(int a, int b) { // Static method
        return a + b;
    }

    public static int subtract(int a, int b) { // Static method
        return a - b;
    }
}

// File: com/example/app/MathApp.java
package com.example.app;

// Import specific static members
import static com.example.math.Calculator.PI;
import static com.example.math.Calculator.add;

// Import all static members from Calculator
// import static com.example.math.Calculator.*;

public class MathApp {
    public static void main(String[] args) {
        double circumference = 2 * PI * 5; // Using PI directly
    }
}
```

```

    System.out.println("Circumference: " + circumference);

    int sum = add(10, 5); // Using add directly
    System.out.println("Sum: " + sum);

    // If 'import static com.example.math.Calculator.*;' was used:
    // int difference = subtract(10, 5);
    // System.out.println("Difference: " + difference);
}
}

// Example with a common static import: System.out
// Instead of System.out.println, we can just use println
// import static java.lang.System.out;

// public class StaticImportExample {
//     public static void main(String[] args) {
//         out.println("Hello from static import!");
//     }
// }

```

4. Constructor Chaining

Constructor chaining is the process of one constructor calling another constructor within the same class or a superclass. This is useful for code reuse and for ensuring that all necessary initialization logic is executed.

a. Within the Same Class (this())

- **Mechanism:** A constructor can call another constructor of the *same class* using this(...).
- **Rule:** The this() call must be the first statement in the constructor.

Example:

Java

```

public class Person {
    private String name;
    private int age;
    private String address;

    // Primary constructor with all fields

```

```

public Person(String name, int age, String address) {
    System.out.println("Calling Person(String, int, String) constructor");
    this.name = name;
    this.age = age;
    this.address = address;
}

// Constructor with name and age, chains to the primary constructor
public Person(String name, int age) {
    this(name, age, "Unknown"); // Calls the Person(String, int, String) constructor
    System.out.println("Calling Person(String, int) constructor");
}

// Constructor with only name, chains to the (name, age) constructor
public Person(String name) {
    this(name, 0); // Calls the Person(String, int) constructor
    System.out.println("Calling Person(String) constructor");
}

public void displayDetails() {
    System.out.println("Name: " + name + ", Age: " + age + ", Address: " + address);
}

public static void main(String[] args) {
    System.out.println("Creating person1:");
    Person person1 = new Person("Alice", 30, "123 Main St");
    person1.displayDetails();
    System.out.println("-----");

    System.out.println("Creating person2:");
    Person person2 = new Person("Bob", 25);
    person2.displayDetails();
    System.out.println("-----");

    System.out.println("Creating person3:");
    Person person3 = new Person("Charlie");
    person3.displayDetails();
    System.out.println("-----");
}
}

```

Output for the above example:

Creating person1:

Calling Person(String, int, String) constructor

Name: Alice, Age: 30, Address: 123 Main St

Creating person2:

Calling Person(String, int, String) constructor

Calling Person(String, int) constructor

Name: Bob, Age: 25, Address: Unknown

Creating person3:

Calling Person(String, int, String) constructor

Calling Person(String, int) constructor

Calling Person(String) constructor

Name: Charlie, Age: 0, Address: Unknown

b. To a Superclass (super())

- **Mechanism:** A constructor can call a constructor of its *superclass* using `super(...)`.
- **Rule:** The `super()` call must be the first statement in the constructor. If you don't explicitly call `super()`, the Java compiler automatically inserts an implicit `super()` call (to the no-argument constructor of the superclass) as the first statement.
- **Importance:** Ensures that the superclass's state is properly initialized before the subclass's constructor executes.

Example (without packages for simplicity, but the concept applies across packages):

Java

```
// Superclass
```

```
class Animal {
```

```
    String species;
```

```
    public Animal(String species) {
```

```
        System.out.println("Calling Animal constructor: " + species);
```

```
        this.species = species;
```

```
    }
```

```
public Animal() {
    this("Unknown"); // Chaining to the other Animal constructor
    System.out.println("Calling no-arg Animal constructor");
}

public void eat() {
    System.out.println("The " + species + " is eating.");
}
}

// Subclass
class Dog extends Animal {
    String breed;

    public Dog(String species, String breed) {
        super(species); // Calls the Animal(String) constructor
        System.out.println("Calling Dog constructor: " + breed);
        this.breed = breed;
    }

    public Dog(String breed) {
        // Implicitly calls super() (no-arg constructor of Animal)
        // If Animal had no no-arg constructor, this would be a compile-time error
        System.out.println("Calling Dog constructor (breed only): " + breed);
        this.breed = breed;
    }

    public void bark() {
        System.out.println("The " + species + " (a " + breed + ") is barking.");
    }
}

public static void main(String[] args) {
    System.out.println("Creating myDog1:");
    Dog myDog1 = new Dog("Canine", "Golden Retriever");
    myDog1.eat();
    myDog1.bark();
    System.out.println("-----");

    System.out.println("Creating myDog2:");
    Dog myDog2 = new Dog("Poodle"); // This will call Animal's no-arg constructor
    myDog2.eat();
    myDog2.bark();
}
```

```

    System.out.println("-----");
}
}

```

Output for the above example:

```

Creating myDog1:
Calling Animal constructor: Canine
Calling Dog constructor: Golden Retriever
The Canine is eating.
The Canine (a Golden Retriever) is barking.
-----
Creating myDog2:
Calling Animal constructor: Unknown
Calling no-arg Animal constructor
Calling Dog constructor (breed only): Poodle
The Unknown is eating.
The Unknown (a Poodle) is barking.
-----

```

Constructor Chaining with Packages

The principle remains the same. The `this()` and `super()` calls govern the constructor chaining, regardless of whether classes are in the same or different packages. The key is that the constructors being called must be accessible.

Java

```

// File: com/example/base/BaseEntity.java
package com.example.base;

public class BaseEntity {
    protected int id;
    protected String creationDate;

    public BaseEntity(int id, String creationDate) {
        System.out.println("BaseEntity(int, String) constructor called.");
    }
}

```

```
this.id = id;
this.creationDate = creationDate;
}

public BaseEntity(int id) {
    this(id, "Today"); // Chaining within the same class
    System.out.println("BaseEntity(int) constructor called.");
}

public void displayBaseInfo() {
    System.out.println("ID: " + id + ", Creation Date: " + creationDate);
}
}

// File: com/example/domain/Product.java
package com.example.domain;

import com.example.base.BaseEntity;

public class Product extends BaseEntity {
    private String name;
    private double price;

    public Product(int id, String creationDate, String name, double price) {
        super(id, creationDate); // Chaining to superclass constructor
        System.out.println("Product(int, String, String, double) constructor called.");
        this.name = name;
        this.price = price;
    }

    public Product(int id, String name, double price) {
        super(id); // Chaining to superclass constructor (BaseEntity(int))
        System.out.println("Product(int, String, double) constructor called.");
        this.name = name;
        this.price = price;
    }

    public void displayProductInfo() {
        displayBaseInfo(); // Inherited protected method
        System.out.println("Name: " + name + ", Price: $" + price);
    }
}

public static void main(String[] args) {
```

```

    System.out.println("Creating product1:");
    Product product1 = new Product(101, "2025-06-21", "Laptop", 1200.00);
    product1.displayProductInfo();
    System.out.println("-----");

    System.out.println("Creating product2:");
    Product product2 = new Product(102, "Mouse", 25.00);
    product2.displayProductInfo();
    System.out.println("-----");
}
}

```

Output for the above example:

```

Creating product1:
BaseEntity(int, String) constructor called.
Product(int, String, String, double) constructor called.
ID: 101, Creation Date: 2025-06-21
Name: Laptop, Price: $1200.0
-----
Creating product2:
BaseEntity(int, String) constructor called.
BaseEntity(int) constructor called.
Product(int, String, double) constructor called.
ID: 102, Creation Date: Today
Name: Mouse, Price: $25.0
-----

```

5. Accessing Protected Variables and Methods Outside the Package

This is a crucial point often misunderstood. While protected members are accessible to subclasses *even if they are in a different package*, there's a specific rule regarding *how* they can be accessed by the subclass outside its own package.

The Rule:

A subclass can access a protected member (field or method) of its superclass *only through an object of the subclass itself* (or an object of a class that inherits from that superclass), *not* through an object of the superclass.

In simpler terms, if ClassB in package2 extends ClassA in package1, ClassB can access ClassA's protected members using this (referring to the ClassB instance) or by creating an instance of ClassB. It **cannot** create an instance of ClassA and then access its protected members.

Example Revisited and Expanded:

Java

```
// File: com/example/vehicles/Vehicle.java
```

```
package com.example.vehicles;
```

```
public class Vehicle {
```

```
    protected String brand;
```

```
    protected int year;
```

```
    public Vehicle(String brand, int year) {
```

```
        this.brand = brand;
```

```
        this.year = year;
```

```
    }
```

```
    protected void displayVehicleInfo() {
```

```
        System.out.println("Vehicle - Brand: " + brand + ", Year: " + year);
```

```
    }
```

```
}
```

```
// File: com/example/app/Truck.java (Subclass in a different package)
```

```
package com.example.app;
```

```
import com.example.vehicles.Vehicle;
```

```
public class Truck extends Vehicle {
```

```
    private double cargoCapacity;
```

```
    public Truck(String brand, int year, double cargoCapacity) {
```

```
        super(brand, year);
```

```
        this.cargoCapacity = cargoCapacity;
```

```
    }
```

```
    public void displayTruckDetails() {
```

```

    // Valid access: Accessing protected members through 'this' (the Truck instance)
    System.out.println("Truck Details:");
    System.out.println("Brand (via protected field): " + this.brand);
    this.displayVehicleInfo(); // Valid: Calling protected method
    System.out.println("Cargo Capacity: " + cargoCapacity + " tons");
}

public static void main(String[] args) {
    Truck myTruck = new Truck("Ford", 2018, 5.0);
    myTruck.displayTruckDetails();

    // --- Demonstrating invalid access attempts ---

    // Attempting to access protected members of a Vehicle object directly
    // from a non-subclass or from a subclass where the object is not 'this'
    Vehicle someVehicle = new Vehicle("Honda", 2022);

    // This would be a compile-time error:
    // System.out.println(someVehicle.brand); // ERROR: Cannot access protected field
    // someVehicle.displayVehicleInfo(); // ERROR: Cannot access protected method

    // This is where the rule becomes apparent:
    // A class in package 'com.example.app' can only access 'protected'
    // members of 'com.example.vehicles.Vehicle' if it is a subclass of 'Vehicle'
    // AND the access is through an object of the subclass type.

    // Even though Truck is a subclass, it cannot access protected members
    // of an arbitrary 'Vehicle' instance (unless that instance is also a Truck).
    // For instance, if you had:
    // Vehicle anotherTruck = new Truck("Volvo", 2023, 10.0);
    // You still couldn't access 'anotherTruck.brand' directly here because
    // 'anotherTruck' is declared as a 'Vehicle' type, not 'Truck' type,
    // and MainApp is not a subclass of Vehicle.
}
}

// File: com/example/app/AppMain.java (A regular class in a different package, not a subclass)
package com.example.app;

import com.example.vehicles.Vehicle; // Still need to import Vehicle to declare its type

public class AppMain {
    public static void main(String[] args) {

```

```
Vehicle generalVehicle = new Vehicle("Nissan", 2021);

// These attempts will result in compile-time errors
// because AppMain is NOT a subclass of Vehicle,
// and thus cannot access Vehicle's protected members outside its package.

// System.out.println(generalVehicle.brand); // ERROR
// generalVehicle.displayVehicleInfo(); // ERROR
}
```

Key Takeaway for Protected Access Outside the Package:

The protected modifier enables a form of "inherited access." It's not about providing general access to objects of the superclass type from outside the package. Instead, it allows subclasses to directly inherit and use those members as part of their own implementation, even if they reside in a different package. The access is effectively tied to the inheritance hierarchy.

This deep dive covers the nuances of Java's access modifiers, package management, and constructor chaining, providing a solid foundation for understanding how to structure and manage your Java code effectively.